

II. 集合框架

一、集合框架顶层接口 (Collection, Map)

1. 什么是Java集合框架 (JCF)？它的主要组成部分是什么？★★★★★

Java集合框架 (JCF) 是一个用来表示和操作对象集合的统一体系结构。它提供了一套标准化的接口、实现和算法，用于高效地存储和处理数据。

其主要组成部分有三个：

- **接口 (Interfaces)**：定义了集合的抽象数据类型 (ADT)，是框架的骨架，规定了集合的功能，如 `List`、`Set`、`Map`。
- **实现 (Implementations)**：接口的具体实现，是可直接使用的数据结构，如 `ArrayList`、`HashSet`、`HashMap`。
- **算法 (Algorithms)**：对集合进行通用操作（如排序、搜索）的多态方法，主要通过 `Collections` 工具类提供。

2. Java集合框架提供了哪些主要接口？它们之间有何关系？★★★★★

Java集合框架有两个独立的顶层接口：`Collection` 和 `Map`。

- **`Collection` 接口**：是处理**单个元素**集合的根接口。它的主要子接口包括：
 - `List`：有序集合，允许元素重复。
 - `Set`：通常无序，不允许元素重复。
 - `Queue`：通常用于实现先进先出 (FIFO) 的队列。
- **`Map` 接口**：是处理**键值对 (key-value)** 映射关系的根接口。

关系总结：`List`、`Set` 和 `Queue` 都继承自 `Collection` 接口。`Map` 接口与 `Collection` 接口体系完全独立，没有继承关系。

3. `Collection` 接口与 `Map` 接口的主要区别是什么？它们是否相互继承？

★★★★★

主要区别在于**数据模型**和**核心用途**：

- **数据模型**：
 - `Collection` 存储的是一组**独立的、单个的对象**。可以想象成一个装物品的“袋子”。
 - `Map` 存储的是一系列**键值对 (key-value pairs)**。可以想象成一本“字典”。
- **核心用途**：
 - `Collection` 用于管理和操作单个元素的集合。
 - `Map` 用于通过唯一的键 (key) 来查找和管理其对应的值 (value)。
- **继承关系**：
 - 它们是JCF中两个**独立的顶层接口**，`Map` **不继承** `Collection`，反之亦然。

4. Java集合框架的优势体现在哪些方面？★★★★☆

JCF的优势主要体现在以下五点：

- 1. **降低编程工作量**：提供了现成的高效数据结构和算法，开发者无需重复造轮子。
- 2. **提高程序性能和质量**：内置实现经过高度优化和严格测试，性能可靠，质量有保证。
- 3. **促进软件重用**：标准化的接口使得不同的数据结构和算法可以轻松地协同工作。
- 4. **实现API间的互操作性**：为不同API之间传递集合数据提供了统一的“语言”。
- 5. **降低学习成本**：开发者只需学习JCF一套标准，就能理解和使用大量依赖它的第三方库。

5. Collection 接口和 Collections 工具类有什么区别？★★★★☆

它们的区别在于类型和角色：

- **Collection**：
 - **类型**：是一个接口 (`java.util.Collection`)。
 - **角色**：是 `List`、`Set` 等集合的父接口，定义了集合的**行为规范**（蓝图）。它定义的是实例方法，由集合对象自身调用。
- **Collections**：
 - **类型**：是一个**工具类** (`java.util.Collections`)。
 - **角色**：提供操作集合的**静态辅助方法**（工具箱），例如排序 (`sort()`)、查找 (`binarySearch()`) 等。这些方法直接通过类名调用。

一句话总结：`Collection` 是规范，`Collections` 是工具。

6. Array 和 Collection 在Java中有什么区别？何时应该使用 Array 而不是 ArrayList？★★★★☆

`Array`（数组）和 `ArrayList`（`Collection` 的一种实现）的主要区别如下：

特性	<code>Array</code>	<code>ArrayList</code>
大小	固定长度，创建后不可变。	动态大小，可自动扩容。
类型	可存原生类型（如 <code>int</code> ）和对象。	只能存对象，原生类型需装箱（如 <code>Integer</code> ）。
性能	访问速度快，无额外开销。	灵活性高，但有动态扩容和装箱/拆箱的开销。
功能	功能有限，方法少。	功能丰富，是JCF一部分，方法多，支持泛型。

何时使用 `Array` 而不是 `ArrayList`？

- 1. 当集合的大小固定且已知，不会改变时。
- 2. 当需要存储原生数据类型（如 `int`）并追求极致性能和内存效率时。
- 3. 当需要处理多维数据时（如 `int`），数组是更自然的选择。

7. Java中的集合类有哪些？如何分类的？ ★★★★★☆

Java中的集合实现类主要根据它们实现的接口来分类：

1. List 实现 (有序、可重复):

- ArrayList: 基于动态数组, **查询快, 增删慢**。
- LinkedList: 基于双向链表, **增删快, 查询慢**。
- Vector: ArrayList 的线程安全版, 已不推荐使用。

2. Set 实现 (通常无序、唯一):

- HashSet: 基于哈希表, **速度最快, 无序**。
- LinkedHashSet: HashSet 子类, **维持插入顺序**。
- TreeSet: 基于红黑树, **元素自动排序**。

3. Map 实现 (键值对):

- HashMap: 基于哈希表, **速度最快, 无序**。
- LinkedHashMap: HashMap 子类, **维持插入顺序或访问顺序**。
- TreeMap: 基于红黑树, **键自动排序**。

4. Queue / Deque 实现 (特定处理顺序):

- PriorityQueue: 优先队列, 按优先级出队。
- ArrayDeque: 高效的双端队列, 可作**栈或队列**使用。

红黑树的平衡规则包括:

1. 每个节点要么是红色, 要么是黑色。
2. 根节点是黑色的。
3. 每个叶子节点 (NIL节点) 是黑色的。
4. 红色节点的子节点必须是黑色的 (不能有两个连续的红色节点)。
5. 从任意节点到其所有叶子的路径上, 必须包含相同数目的黑色节点 (即黑色高度相等)。

8. 你能说出几种集合的排序方式？ ★★★★★☆

主要有三种技术, 尤其是在Java 8之后:

1. Collections.sort(List): 使用**工具类**对 List **原地排序**。
2. list.sort(Comparator) (Java 8+): List 接口的默认方法, 也是**原地排序**, 对 ArrayList 性能更好。
3. stream.sorted() (Java 8+): Stream API的中间操作, **不修改原始集合**, 返回一个已排序的**新流**。

二、List 接口及其核心实现类 (ArrayList, LinkedList)

1. List 接口特性与比较

1. List 接口的特点是什么？它与 Set 接口有何不同？ ★★★★★

- **List 接口特点:**
 1. **有序性 (Ordered):** 元素按插入顺序存储, 可通过索引访问。
 2. **可重复性 (Allows Duplicates):** 可以包含相同的元素。
- **与 Set 接口区别:**
 - **List:** 有序, 可重复。
 - **Set:** 通常无序 (`HashSet`), 且绝对不重复。

2. **ArrayList、LinkedList 与 Vector 的区别? ★★★★★**

- **ArrayList:**
 - **底层:** 动态数组 (`Object[]`)。
 - **性能:** 查改快 ($O(1)$), 增删慢 ($O(n)$)。
 - **线程安全:** 否。
- **LinkedList:**
 - **底层:** 双向链表。
 - **性能:** 查改慢 ($O(n)$), 首尾增删快 ($O(1)$)。
 - **线程安全:** 否。
- **Vector:**
 - **底层:** 动态数组。
 - **性能:** 因方法加锁 (`synchronized`), 性能比 `ArrayList` 差。
 - **线程安全:** 是。但已被淘汰, 并发场景应使用 `CopyOnWriteArrayList`。

3. 请详细比较 `ArrayList` 和 `LinkedList` 的异同点 (底层数据结构、各种操作的性能、内存使用、线程安全性)。它们各自适用于什么场景? ★★★★★

特性	<code>ArrayList</code>	<code>LinkedList</code>
底层数据结构	动态数组 (<code>Object[]</code>)	双向链表 (<code>Node</code> 节点)
随机访问 (<code>get</code>)	极快, $O(1)$	慢, $O(n)$, 需遍历
插入/删除 (中间)	慢, $O(n)$, 需移动元素	快, $O(1)$ (若有迭代器定位)
内存占用	较小, 连续内存	较大, 需额外存前后指针
线程安全性	否	否

- **`ArrayList` 适用场景:** 读多写少。需要频繁进行随机访问的场景。
- **`LinkedList` 适用场景:** 写多读少。需要频繁在列表首尾进行增删的场景 (如用作队列、栈)。

2. ArrayList 详解

1. ArrayList 的底层数据结构是什么？它是如何实现动态扩容的？★★★★★

- **底层数据结构:** ArrayList 的底层是一个**动态数组**，在源码中是 `transient Object[] elementData`；。
- **动态扩容:** 当向 ArrayList 添加元素，而内部数组容量不足时，它会创建一个**更大的新数组**，并将旧数组中的所有元素**复制**到新数组中。这个过程对用户是透明的。

2. ArrayList 的扩容机制具体是怎样的？新容量是如何计算的？★★★★☆

1. **触发时机:** 调用 `add` 方法时，如果发现当前容量 `size + 1` 大于 `elementData.length`，则触发扩容。
2. **计算新容量:** 新容量是旧容量的 **1.5 倍**。源码为：`int newCapacity = oldCapacity + (oldCapacity >> 1);` (`>> 1` 是右移一位，等效于除以2)。
3. **特殊情况:** 如果是首次添加元素（数组为空），则会初始化一个默认容量为10的数组。如果扩容后的新容量仍然小于所需最小容量，则直接使用所需最小容量。

3. ArrayList 的序列化是怎么实现的？★★★☆☆

ArrayList 通过自定义序列化逻辑来优化存储。

1. `elementData` 数组被 `transient` 关键字修饰，这意味着它**不会被默认的序列化机制序列化**。
2. ArrayList 重写了 `writeObject()` 和 `readObject()` 方法。
3. 在 `writeObject()` 中，它只序列化 `size` 和**实际存储的元素**，而不是整个 `elementData` 数组。这样可以避免序列化数组中的 `null` 元素，从而节省空间。

4. ArrayList 的 subList 方法有什么需要注意的地方吗？★★★★☆

`subList()` 返回的不是一个全新的 ArrayList，而是一个**原 ArrayList 的视图**。这是个大坑！

1. **非结构性修改同步:** 对 `subList` 的修改（如 `set`）会直接反映到原 `List` 上，反之亦然。
2. **结构性修改导致异常:** **绝对不能**在获取 `subList` 视图后，对**原 List** 进行结构性修改（如 `add` 或 `remove`）。否则，当操作这个 `subList` 时，会立即抛出 `ConcurrentModificationException`。

3. LinkedList 详解

1. LinkedList 的底层数据结构是什么？它与 ArrayList 在底层实现上有何不同？★★★★★

- **底层数据结构:** LinkedList 的底层是一个**双向链表 (Doubly-linked list)**。
- **与 ArrayList 的不同:**
 - ArrayList: 使用**连续的内存空间**（数组）存储数据。
 - LinkedList: 使用**不连续的内存空间**，通过节点内的指针将所有元素链接起来。

2. LinkedList 作为一个双向链表，其 Node 节点包含了哪些信息？★★★★☆

LinkedList 内部的私有静态类 `Node<E>` 包含了三个核心信息：

1. `E item`: 当前节点存储的**数据元素**。
2. `Node<E> next`: 指向**下一个节点**的引用。
3. `Node<E> prev`: 指向**上一个节点**的引用。

4. Vector 与 Stack

1. `Vector` 和 `Stack` 是什么？它们与 `ArrayList` 和 `LinkedList` 有何区别？它们是线程安全的吗？为什么？★★★★☆

- `Vector`: 是一个古老的 `List` 实现，可以看作 `ArrayList` 的前身。
- `Stack`: 继承自 `Vector`，实现了经典“后进先出”(LIFO)的栈结构。
- **线程安全**: 它们都是线程安全的。
- **原因**: 它们内部几乎所有公开方法都使用了 `synchronized` 关键字进行同步，保证了在同一时刻只有一个线程能访问这些方法。但由于锁的粒度太大，导致性能低下，因此不推荐使用。
 - **替代品**: 栈操作推荐使用更高效的 `ArrayDeque`。

5. List 与 Array 转换

1. 如何将 `ArrayList` 转换为 `Array`，反之亦然？在使用 `Arrays.asList()` 时需要注意什么？★★★★☆

- **List 转 Array**: 使用 `list.toArray(new String[0])`。传入一个指定类型的空数组是最高效的方式。
- **Array 转 List**: 使用 `Arrays.asList(array)`。
- **Arrays.asList() 的注意事项 (重要)**:
 1. 返回的 `List` **不是** `java.util.ArrayList`，而是 `Arrays` 的一个内部类。
 2. 这个返回的 `List` 是**定长的**，它仍然由原始数组支持。因此，**不能**对其进行 `add` 或 `remove` 操作，否则会抛出 `UnsupportedOperationException`。
 3. 修改原始数组的元素，会同步影响到转换后的 `List`。

三、Set 接口及其实现类 (HashSet, LinkedHashSet, TreeSet)

1. Set 接口特性与比较

1. `Set` 接口的特点是什么？它与 `List` 接口的主要区别是什么？★★★★★

- **Set 接口**的特点:
 1. **唯一性 (Uniqueness)**: 集合中**不允许包含重复的元素**。这是 `Set` 最核心的特征。
 2. **无序性 (Unordered)**: 大多数 `Set` 实现（如 `HashSet`）不保证元素的存储和迭代顺序。
- **与 List 接口的主要区别**:
 - **重复性**: `Set` **不允许**重复，而 `List` **允许**重复。
 - **顺序性**: `Set` 通常是**无序的**，而 `List` 是**有序的**（按插入顺序）。
 - **索引**: `Set` 没有索引，不能通过下标访问元素。`List` 有索引，支持随机访问。

2. `Set` 是如何保证元素不重复的？★★★★★

`Set` 的唯一性是依赖于元素的 `hashCode()` 和 `equals()` 这两个方法来保证的。

当向 `Set` 中添加一个新元素时（例如 `HashSet`）：

1. **计算哈希码**: 首先调用该元素的 `hashCode()` 方法，计算出哈希值，并根据哈希值确定该元素在哈希表中的存储位置。
2. **检查冲突**:

- 如果该位置上没有任何元素，则直接将新元素存入。
- 如果该位置上已经有其他元素（哈希冲突），则会遍历该位置上的所有元素。

3. 调用 `equals()` 比较: 依次调用新元素与已有元素的 `equals()` 方法进行比较。

- 如果 `equals()` 返回 `true`，说明元素已存在，`add` 操作失败，新元素不会被添加。
- 如果遍历完所有元素，`equals()` 都返回 `false`，说明元素不存在，则将新元素添加到该位置的链表（或红黑树）中。

结论: 两个对象 `hashCode()` 相等，`equals()` 也返回 `true`，`Set` 才会认为它们是同一个对象，从而保证了唯一性。

3. 详细比较 `HashSet`、`LinkedHashSet` 和 `TreeSet` 的差异。它们各自适用于什么场景？

★★★★★

特性	<code>HashSet</code>	<code>LinkedHashSet</code>	<code>TreeSet</code>
底层数据结构	<code>HashMap</code>	<code>LinkedHashMap</code>	<code>TreeMap</code>
元素顺序	无序	插入顺序	排序 (自然顺序或自定义)
性能	最高，增删查改都是 $O(1)$	略低于 <code>HashSet</code> (维护链表)	最低，增删查改都是 $O(\log n)$
<code>null</code> 元素	允许一个 <code>null</code>	允许一个 <code>null</code>	不允许 (默认情况)
线程安全	否	否	否

适用场景:

- `HashSet`: 追求最高性能，不关心元素顺序。用于简单地存储不重复的元素。
- `LinkedHashSet`: 需要 `Set` 的唯一性，同时希望保持元素的插入顺序。
- `TreeSet`: 需要一个时刻保持有序状态的唯一元素集合。例如，对数据进行去重后排序。

2. `HashSet` 详解

1. `HashSet` 的底层数据结构是什么？它如何存储元素并判断元素的唯一性？★★★★★

- **底层数据结构:** `HashSet` 的底层实际上是一个 `HashMap`。
- **存储与唯一性判断:**
 - 当你向 `HashSet` 中 `add(element)` 时，它实际上是调用了内部 `HashMap` 的 `put(element, PRESENT)` 方法。
 - `HashSet` 将要添加的元素作为 `HashMap` 的键 (key)。
 - `HashMap` 的值 (value) 则是一个固定的、静态的虚拟对象 `PRESENT`。
 - 因为 `HashMap` 的键本身就是唯一的，所以当尝试添加重复元素时，`put` 方法会覆盖旧值（但因为值都是同一个 `PRESENT`，所以等于什么都没做），并返回旧值（非 `null`），`HashSet` 的 `add` 方法根据这个返回值判断出元素已存在，并返回 `false`。

3. TreeSet 详解

1. TreeSet 的底层数据结构是什么？它如何保证元素的有序性？★★★★★

- **底层数据结构:** TreeSet 的底层是一个 TreeMap。
- **有序性保证:**
 - 与 HashSet 类似, TreeSet 将元素作为其内部 TreeMap 的键 (key)。
 - TreeMap 的底层是一个**红黑树 (Red-Black Tree)**, 它是一种自平衡的二叉查找树, 能保证其键始终处于有序状态。
 - 每当添加一个新元素时, TreeMap 会根据比较结果将其放置在树的正确位置, 以维持排序。比较的依据是:
 1. **自然排序:** 元素类需要实现 Comparable 接口。
 2. **自定义排序:** 在创建 TreeSet 时传入一个 Comparator 对象。

4. LinkedHashMap 详解

1. LinkedHashMap 的底层数据结构是什么？它是如何维护元素的插入顺序的？★★★★☆

- **底层数据结构:** LinkedHashMap 的底层是一个 LinkedHashMap。
- **插入顺序维护:**
 - LinkedHashMap 继承自 HashMap, 它在 HashMap (哈希表) 的基础上, 额外维护了一个贯穿所有条目的**双向链表**。
 - 这个双向链表负责记录元素的**插入顺序**。
 - 每当有新元素插入时, 除了在哈希表中定位, 还会将这个新条目链接到双向链表的末尾。
 - 因此, 当遍历 LinkedHashMap 时, 实际上是按照这个双向链表的顺序进行遍历, 从而实现了“先进先出”的迭代顺序。

5. Set 与 null 元素

1. 能否在 TreeSet 或 HashSet 中添加 null 元素？请解释原因。★★★★☆

- HashSet / LinkedHashMap: **可以添加一个 null 元素**。因为它们底层的 HashMap 和 LinkedHashMap 允许一个 null 作为键。
- TreeSet: **默认情况不可以添加 null 元素**。
 - **原因:** TreeSet 需要对元素进行比较来排序。当尝试添加第一个 null 元素时, 在后续添加非 null 元素时, 会调用 compareTo() 或 compare() 方法与 null 进行比较, 这通常会抛出 NullPointerException。
 - **例外:** 如果在创建 TreeSet 时, 提供了一个能够处理 null 值的特殊 Comparator, 那么就可以添加 null 元素。但这在实际中非常罕见。

四、Map 接口及其核心实现类（HashMap, LinkedHashMap, TreeMap）

1. Map 接口特性与比较

1. Map 接口的目的是什么？它与 Collection 接口有何关系？它与 List 和 Set 又有何不同？★★★★★

- **目的:** Map 接口用于存储**键值对 (Key-Value)** 的映射关系。它提供了一种通过唯一的**键 (Key)** 来快速查找、更新或删除对应**值 (Value)** 的机制。
- **与 Collection 的关系:** Map 与 Collection 是Java集合框架中两个**独立的顶层接口**，它们之间**没有继承关系**。
- **与 List 和 Set 的不同:**
 - **数据结构:** List 和 Set 存储的是**单个元素**的集合，而 Map 存储的是**键值对**。
 - **核心关注点:** List 关注**索引和顺序**，Set 关注**元素的唯一性**，而 Map 关注**键与值之间的映射关系**。

2. 比较 LinkedHashMap 和 TreeMap 在元素顺序、底层数据结构、性能以及对 null 键处理方面的差异。它们各自的典型应用场景是什么？★★★★☆

特性	LinkedHashMap	TreeMap
底层数据结构	哈希表 + 双向链表	红黑树
元素顺序	维护 插入顺序 或 访问顺序	维护键的 自然排序 或 自定义排序
性能	增删查改通常为 O(1)	增删查改均为 O(log n)
null 键处理	允许 一个 null 键	不允许 null 键（因为无法比较）
典型应用场景	1. 需要保证迭代顺序与插入顺序一致的场景。 2. 实现 LRU (最近最少使用) 缓存。	1. 需要一个键始终有序的映射。 2. 需要进行范围查找的场景。

2. HashMap 详解

1. HashMap 的底层数据结构是什么？JDK1.7 和 JDK1.8 在底层实现上有什么区别和改进？★★★★★

- **底层数据结构:**
 - **JDK 1.7:** 数组 + 链表。
 - **JDK 1.8:** 数组 + 链表 / 红黑树。
- **区别和改进:**
 1. **哈希冲突解决方式:** JDK 1.7 中，哈希冲突时，元素都以链表形式存储。JDK 1.8 在链表长度超过**阈值 (8)** ***时，会将链表*转换为红黑树**，以优化查询性能。
 2. **插入方式:** JDK 1.7 使用**头插法**，在多线程并发扩容时可能导致链表形成**环形**，从而引发死循环。JDK 1.8 改用**尾插法**，解决了这个问题。
 3. **哈希算法:** JDK 1.8 的哈希算法比 1.7 的更简化和高效。
 4. **扩容机制:** JDK 1.8 在扩容时，对元素位置的计算做了优化，使得元素要么留在原索引，要么移动到 **原索引 + 旧容量** 的位置，提高了效率。

2. 解释 HashMap 的内部工作原理（哈希、桶、索引计算、哈希冲突）。★★★★★

1. **哈希 (Hashing):** 当调用 `put(key, value)` 时, `HashMap` 首先调用 `key` 的 `hashCode()` 方法, 并将返回的哈希码通过一个扰动函数 (`hash()` 方法) 进行二次处理, 以获得更好的散列效果。
2. **桶 (Bucket):** `HashMap` 内部维护一个数组 (`Node[] table`), 数组的每个位置被称为一个“桶”。
3. **索引计算:** 使用处理后的哈希值与 (`数组长度 - 1`) 进行**按位与运算 (&)** 来计算出键值对应该存放的桶的索引。 `hash & (n-1)` 的效果等价于 `hash % n`, 但位运算效率更高。
4. **哈希冲突 (Collision):** 如果计算出的索引位置已经有其他键值对存在, 就发生了哈希冲突。
`HashMap` 会在该桶中以链表或红黑树的形式继续存储新的键值对。

3. 解决哈希冲突的方法有哪些? HashMap 使用了哪种或哪几种? ★★★★★☆

- **主要方法:**
 1. **开放定址法:** 如果发生冲突, 就去寻找下一个空的散列地址。
 2. **再哈希法:** 使用另一个哈希函数计算新的地址。
 3. **链地址法 (拉链法):** 在冲突位置建立一个链表 (或其它数据结构), 将所有冲突的元素都链接在该位置上。
- **HashMap 的选择:** `HashMap` 使用的是**链地址法**。在 JDK 1.8 中, 当链表过长时, 会进一步演化为红黑树。

4. 为什么 HashMap 在解决哈希冲突时, 链表长度达到阈值后会转换为红黑树? 其转换阈值 (8) 和回退阈值 (6) 是基于什么考量? ★★★★★

- **为什么转换:**
 - **性能优化:** 当哈希冲突严重时, 链表的查询时间复杂度会退化到 $O(n)$ 。而红黑树作为一种自平衡二叉查找树, 其查询时间复杂度稳定在 $O(\log n)$ 。将链表转换为红黑树是为了**优化极端情况下的查询性能**。
- **阈值考量:**
 - **转换阈值 (TREEIFY_THRESHOLD = 8):** 这个值的选择是基于**泊松分布**。在一个设计良好的哈希函数下, 单个桶中元素数量达到8个的概率极低。一旦发生, 就说明可能存在哈希攻击或 `hashCode` 实现很差, 此时有必要进行树化。选择8是在时间和空间成本之间做的权衡。
 - **回退阈值 (UNTREEIFY_THRESHOLD = 6):** 当从红黑树中移除元素, 使其节点数减少到6时, 会退化为链表。设置这个值是为了**避免在阈值8附近频繁地进行树化和反树化操作**, 这提供了一个缓冲地带 (Hysteresis)。

5. HashMap 的默认加载因子 (Load Factor) 是多少? 为什么是0.75? 它对 HashMap 的性能和内存使用有何影响? ★★★★★

- **默认值:** `HashMap` 的默认加载因子是 **0.75**。
- **为什么是0.75:** 这是在**时间成本 (查找开销) * 和 * 空间成本 (内存占用) * 之间寻求的一个*理想平衡点**。
 - 加载因子过高 (如1.0): 节省了空间, 但哈希冲突的概率大大增加, 导致查找性能下降。
 - 加载因子过低 (如0.5): 减少了哈希冲突, 查找性能高, 但浪费了大量空间, 因为 `HashMap` 会更频繁地扩容。
 - 0.75这个值是根据大量实验数据得出的一个经验值, 能较好地平衡两者。
- **影响:** 加载因子决定了 `HashMap` 何时进行扩容 (resize)。扩容的触发条件是: `size > capacity * loadFactor`。

6. 详细解释 HashMap 的 put 方法流程。JDK1.7 的 HashMap 在多线程环境下扩容时可能出现死循环问题的原因是什么？JDK1.8 如何解决？★★★★★

- **put 流程 (JDK 1.8):**

1. 判断内部数组 `table` 是否为空，如果为空则执行 `resize()` 初始化。
2. 计算键 `key` 的哈希值，并据此计算出在数组中的索引 `i`。
3. 如果索引 `i` 处的桶为空，直接新建一个节点 `Node` 存入。
4. 如果桶不为空：
 - a. 检查桶的第一个节点的键是否与 `key` 相同，如果相同则直接覆盖 `value`。
 - b. 如果第一个节点不同，判断该桶是红黑树还是链表。
 - c. 如果是红黑树，则调用红黑树的插入方法。
 - d. 如果是链表，则遍历链表，如果找到相同的键则覆盖 `value`，如果遍历到末尾仍未找到，则在链表尾部插入新节点。插入后，检查链表长度是否达到树化阈值8，如果达到则将链表转换为红黑树。
5. 插入成功后，判断 `size` 是否超过 `capacity * loadFactor`，如果超过则触发 `resize()` 扩容。

- **JDK 1.7 死循环原因:**

- `HashMap` 在扩容时，需要将旧数组中的元素重新哈希到新数组中。JDK 1.7 在转移链表元素时使用的是**头插法**，这会导致链表的顺序**反转**。
- 在多线程环境下，如果两个线程同时对同一个 `HashMap` 进行扩容，一个线程可能在另一个线程完成反转之前拿到链表头，导致A指向B，而B又被另一个线程反转后指向A，形成一个**循环链表**。当后续 `get` 操作遍历这个链表时，就会陷入死循环。

- **JDK 1.8 解决方案:**

- JDK 1.8 在扩容时改用**尾插法**，并巧妙地设计了重哈希过程。它在转移元素时能保持链表原来的顺序，从而**从根本上避免了链表反转和形成环路的可能性**。

7. 通常用什么作为 HashMap 的键？一个类要作为 Map 键必须满足什么条件？为什么强烈建议键类是不可变的？★★★★★

- **通常用什么作键:** 通常使用 `String`、`Integer` 等**不可变类**作为键。

- **作为键的条件:**

1. 必须正确地**重写** `hashCode()` 和 `equals()` 方法。
2. `hashCode()` 契约：两个对象如果 `equals()` 相等，它们的 `hashCode()` 必须相等。
3. `equals()` 契约：必须满足自反性、对称性、传递性、一致性。

- **为什么建议键不可变:**

- **保证能取回值:** 如果键对象在放入 `HashMap` 后，其内部状态发生改变，导致其 `hashCode()` 值也发生改变，那么 `HashMap` 将无法根据原来的 `hashCode()` 计算出正确的存储位置，从而**导致无法找到这个键值对**，就好像它丢失了一样。
- **保持一致性:** 不可变性保证了对象的 `hashCode()` 在其生命周期内始终如一，确保了 `HashMap` 的行为是可预测和正确的。

8. HashMap 的hash方法是如何实现的? ★★★★★☆

JDK 1.8 中的 `hash()` 方法非常简洁: `static final int hash(Object key) { int h; return (key == null) ? 0 : (h = key.hashCode()) ^ (h >> 16); }`

- 实现步骤:

1. 获取键 `key` 原始的 `hashCode()` 值 `h`。
2. 将 `h` **无符号右移16位**, 得到其高16位。
3. 将原始的 `h` (包含高16位和低16位) 与右移后的高16位进行**异或 (^) 运算**。

- **目的:** 这样做是为了让哈希码的**高16位也参与到最终的索引计算中**, 增加了随机性, 使得哈希值分布更均匀, 从而减少哈希冲突。这对于那些 `hashCode()` 实现得不好, 变化只集中在高位的类尤其有效。

9. HashMap 的数据结构是怎样的? ★★★★★☆

`HashMap` 的核心数据结构是一个 `Node<K,V>[] table`, 即一个 `Node` 类型的**数组**。

- 这个数组的每个元素都是一个数据节点, 可以看作一个“桶” (bucket) 。
- 当发生哈希冲突时, 同一个桶中的多个节点会以**链表**的形式组织起来。
- 当链表长度超过阈值 (8) 并且数组容量大于64时, 这个链表会转化为一棵**红黑树**以提高查询效率。

所以, `HashMap` 的数据结构可以总结为: **数组 + (链表 或 红黑树)** 。

10. HashMap 的容量为什么是2的幂次方? 如何保证? ★★★★★☆

- 为什么是2的幂次方:

- **为了高效计算索引。** 当容量 `n` 是2的幂次方时, 取模运算 `hash % n` 就可以用更高效的位运算 `hash & (n-1)` 来代替。计算机执行位运算的速度远快于取模运算。

- 如何保证:

- `HashMap` 内部有一个 `tableSizeFor(int cap)` 方法。无论你传入什么初始容量, 这个方法都会返回**大于或等于该容量的、最小的2的幂次方数**。例如, 如果你指定初始容量为10, `HashMap` 实际会创建容量为16的数组。

11. HashMap 是如何扩容的? ★★★★★☆

扩容 (`resize()`) 是一个重新分配内部数组并重新安置所有元素的过程。

1. **触发时机:** 当 `HashMap` 的 `size` (元素数量) 超过 `capacity * loadFactor` (容量 * 加载因子) 时触发。
2. **扩容大小:** 通常, 新容量是**旧容量的两倍**。
3. 扩容过程 (JDK 1.8):
 - a. 创建一个新的、容量为原先两倍的数组。
 - b. 遍历旧数组中的每一个桶。
 - c. 如果桶中只有一个节点, 直接计算新索引并放入新数组。
 - d. 如果桶中是链表或红黑树, 则遍历所有节点。由于容量是2的幂次方, 扩容后, 每个节点的新位置要么在原索引, 要么在**原索引 + 旧容量**的位置。通过 `(e.hash & oldCap) == 0` 这个巧妙的判断, 就可以将一条链上的节点高效地拆分到两个新位置上, 而无需对每个节点都重新计算完整的哈希索引。

12. 为什么在JDK8中HashMap要转成红黑树? ★★★★★☆

这是为了应对极端情况下的性能退化问题。

- 理想情况下, `HashMap` 的增删查改时间复杂度都是 $O(1)$ 。
- 但是, 如果大量元素的 `hashCode()` 值相同或很接近, 会导致它们被映射到同一个桶中, 形成一条长长的链表。在这种最坏情况下, `HashMap` 的操作时间复杂度会退化成 $O(n)$, 和 `ArrayList` 一样慢。
- 引入红黑树后, 即使发生严重的哈希冲突, 单个桶内的查询时间复杂度也能保持在 $O(\log n)$, 从而保证了 `HashMap` 在最坏情况下的性能。

13. HashMap 的remove方法是如何实现的? ★★★☆☆

`remove(key)` 方法的实现逻辑与 `put` 和 `get` 类似:

- 计算 `key` 的哈希值并定位到对应的桶。
- 如果桶为空, 直接返回 `null`。
- 如果桶不为空, 遍历链表或在红黑树中查找与 `key` 相匹配的节点。
- 如果找到了节点, 就将其从链表或红黑树中删除 (通过修改前后节点的指针引用或执行树的删除操作), 并返回该节点对应的 `value`。
- 如果遍历完整个桶都没找到匹配的 `key`, 则返回 `null`。

14. HashMap 在并发场景中有什么问题? ★★★★★

`HashMap` 是非线程安全的, 在并发场景下直接使用会引发严重问题:

- 数据覆盖/丢失 (JDK 1.8):** 多线程同时执行 `put` 操作, 如果没有正确的同步, 后一个线程的写入可能会覆盖前一个线程的写入, 导致数据丢失。
- 死循环 (JDK 1.7):** 在 JDK 1.7 中, 多线程并发扩容时, 由于头插法会导致链表反转, 可能形成循环链表, 导致 `get` 操作时 CPU 100% 死循环。
- 数据不一致:** 一个线程正在遍历 `HashMap` 时, 另一个线程修改了其结构 (`put` 或 `remove`), 会导致 `ConcurrentModificationException`。

解决方案:

- 使用 `Collections.synchronizedMap(new HashMap<>())` 包装。
- 使用 `Hashtable` (不推荐)。
- 使用 `ConcurrentHashMap` (推荐的最佳实践)。

15. HashMap的容量设置多少合适? ★★★☆☆

为了获得最佳性能, 应尽量避免或减少 `resize()` 操作。

- 经验法则:** 如果你可以预估将要存储的元素数量 `n`, 那么一个合理的初始容量设置是 $(\text{int})(n / 0.75f) + 1$ 。
- 例如, 如果你要存储1000个元素, 初始容量应设为 $(\text{int})(1000 / 0.75) + 1 = 1334$ 。
`HashMap` 会自动将其调整为最近的2的幂次方, 即2048。这样可以在所有元素插入完成前, 基本保证不会发生扩容。

3. Hashtable 详解

1. 详细比较 HashMap 和 Hashtable。为什么 Hashtable 不再推荐使用？★★★★★

特性	HashMap	Hashtable
线程安全	非线程安全，效率高	线程安全，方法都用 <code>synchronized</code> 修饰，效率低
null 值处理	允许一个 null 键和多个 null 值	不允许 null 键和 null 值，会抛 <code>NullPointerException</code>
继承体系	继承自 <code>AbstractMap</code>	继承自 <code>Dictionary</code> (一个遗留类)
迭代器类型	使用 <code>Iterator</code> (fail-fast)	同时支持 <code>Iterator</code> (fail-fast) 和 <code>Enumeration</code> (fail-safe)
效率	单线程下高	因为同步开销，低
初始容量/扩容	初始16，按2倍扩容	初始11，按 $2 * old + 1$ 扩容
底层数据结构	JDK 1.8 为数组+链表/红黑树	数组+链表

为什么不再推荐使用 Hashtable？

- 性能低下:** `Hashtable` 使用 `synchronized` 锁住了几乎所有公共方法，锁的粒度太大。这意味着在同一时间只有一个线程能访问它，导致在并发环境下的性能非常差。
- 有更好的替代品:** `java.util.concurrent.ConcurrentHashMap` 提供了更先进、更高效的并发控制（通过分段锁或CAS），在保证线程安全的同时，性能远超 `Hashtable`。对于非并发场景，`HashMap` 是更好的选择。

4. 特殊 Map 实现类

1. 什么是 IdentityHashMap？它与 HashMap 在键的比较上有何不同？其典型用途是什么？★★★☆☆

- 是什么:** 一个特殊的 `Map` 实现，它在比较键时违反了 `Map` 的通用约定。
- 与 HashMap 的不同:** `HashMap` 使用键的 `equals()` 方法来判断两个键是否相等。而 `IdentityHashMap` 使用引用相等性 (`==`) 来判断，即只有两个引用指向堆中同一个对象时，才被认为是同一个键。
- 典型用途:**
 - 对象序列化或深拷贝时，用于跟踪已经处理过的对象，防止无限循环。
 - 需要将元数据附加到对象上，且必须区分不同对象实例的场景。

2. 什么是 WeakHashMap？它如何处理键的生命周期？其典型用途是什么？★★★☆☆

- 是什么:** 一个键为“弱引用”的 `Map` 实现。
- 如何处理键:** 当 `WeakHashMap` 的某个键不再有任何强引用指向它时（即只剩下 `WeakHashMap` 内部的弱引用），垃圾回收器（GC）就可以回收这个键对象。在下次 `Map` 被操作时，这些“已死”的键值对就会被自动移除。
- 典型用途:**

- **实现缓存。**当外部不再使用某个对象时，`WeakHashMap` 可以自动清理对应的缓存条目，避免内存泄漏。例如，Tomcat 的 `ConcurrentCache` 就利用了它。

3. 什么是 EnumMap？其底层实现原理是什么？它相比于其他 Map 实现有哪些优势？ ★★★☆☆

- **是什么：**一个专门为**枚举类型键**设计的高性能 `Map` 实现。
- **底层原理：**`EnumMap` 内部不使用哈希表，而是使用一个**数组**作为其底层数据结构。数组的大小就是该枚举类型常量的数量，每个枚举常量通过其 `ordinal()` 值（从0开始的整数）直接映射到数组的索引。
- **优势：**
 1. **极高的性能：**所有操作都是 $O(1)$ ，且由于没有哈希计算和冲突处理，其常数时间开销远小于 `HashMap`。
 2. **内存占用小：**内部实现非常紧凑。
 3. **保证顺序：**迭代器严格按照枚举常量的自然顺序（定义顺序）返回键。

5. Map 遍历方式

1. 在Java中遍历 Map 有哪些常用的方法？它们各自适用于什么场景？哪种方法是最高效的？★★★★☆

1. 遍历 `entrySet()`

- `for (Map.Entry<K, V> entry : map.entrySet()) { ... }`
- **适用场景：**需要同时访问键和值的场景。
- **效率：最高效。**因为它只遍历一次，就可以从 `Entry` 对象中直接获取键和值。

2. 遍历 `keySet()`

- `for (K key : map.keySet()) { V value = map.get(key); ... }`
- **适用场景：**只需要访问键，或者需要通过键来操作 `Map`（如 `remove`）的场景。
- **效率：**当需要值时效率较低，因为对每个键都进行了一次 `get()` 操作，相当于进行了第二次查找。

3. 只遍历 `values()`

- `for (V value : map.values()) { ... }`
- **适用场景：**只需要访问值，不关心键。

4. 使用 `forEach` 和 `Lambda (Java 8+)`

- `map.forEach((key, value) -> { ... });`
- **适用场景：**语法简洁，适合函数式编程风格。
- **效率：**其内部实现通常也是基于 `entrySet()`，因此效率很高。

结论：当需要同时处理键和值时，**遍历 `entrySet()` 是最高效、最推荐的方式。**

五、Queue / Deque 接口及其实现类

1. Queue 接口特性

1. Queue 接口的特点是什么？它主要遵循什么原则？其常用的添加、移除和检查方法有哪些？

★★★★☆

- **特点:** `Queue` (队列) 是为在处理前持有元素而设计的集合。
- **原则:** 通常遵循**先进先出 (FIFO, First-In-First-Out)** 原则, 即最先进入队列的元素最先被移出。
- **常用方法:** `Queue` 为每种操作提供了两种形式的方法: 一种在失败时**抛出异常**, 另一种则返回**特殊值** (`null` 或 `false`) 。

操作类型	失败时抛出异常	失败时返回特殊值
添加 (Insert)	<code>add(e)</code>	<code>offer(e)</code>
移除 (Remove)	<code>remove()</code>	<code>poll()</code>
检查 (Examine)	<code>element()</code>	<code>peek()</code>

记忆口诀: `offer`、`poll`、`peek` 是更温和的选择, 推荐在生产代码中使用以避免不必要的异常。

2. `LinkedList` 如何作为 `Queue` 或 `Deque` 使用? ★★★☆☆

`LinkedList` 类同时实现了 `List`、`Queue` 和 `Deque` 接口, 这使得它非常灵活。

- **作为 `Queue` (队列):** 利用其在链表尾部添加、头部移除的高效性。
 - 入队: 使用 `offer(e)` 或 `add(e)`。
 - 出队: 使用 `poll()` 或 `remove()`。
- **作为 `Deque` (双端队列):** 提供了在两端进行操作的方法。
 - 头端操作: `addFirst()`, `removeFirst()`, `getFirst()` 等。
 - 尾端操作: `addLast()`, `removeLast()`, `getLast()` 等。

2. `PriorityQueue` 详解

1. 什么是 `PriorityQueue`? 它的底层数据结构是什么? 它是如何实现优先级排序的? ★★★☆☆

- **是什么:** `PriorityQueue` 是一个**优先队列**, 它不遵循 FIFO 原则, 而是根据元素的**优先级**进行出队。优先级最高的元素最先被移出。
- **底层数据结构:** 底层是一个**二叉堆** (具体来说是最小堆), 通过数组实现。
- **实现优先级排序:**
 1. **自然排序:** 存入 `PriorityQueue` 的元素类需要实现 `Comparable` 接口。
 2. **自定义排序:** 在创建 `PriorityQueue` 时传入一个 `Comparator` 对象。

2. 简述 `PriorityQueue` 中的“上浮”和“下沉”操作。★★★★☆

“上浮” (`siftUp`) 和“下沉” (`siftDown`) 是维持二叉堆性质的核心操作。

- **上浮 (`siftUp`):** 当向堆中**添加**一个新元素时, 会先将其置于数组末尾。然后, 将该元素与其父节点比较, 如果它的优先级更高 (值更小), 则与父节点交换位置。这个过程不断重复, 直到该元素到达正确的位置 (其优先级不再高于父节点) 或成为根节点。
- **下沉 (`siftDown`):** 当从堆顶**移除**一个元素后, 会把数组末尾的元素放到堆顶。然后, 将这个新堆顶元素与其子节点比较, 如果它的优先级低于子节点 (值更大), 则与优先级最高的子节点交换位置。这个过程不断重复, 直到该元素“下沉”到正确的位置 (其优先级不再低于任何子节点) 或成为叶子节点。

3. BlockingQueue 详解

1. 什么是 `BlockingQueue`？它的目的是什么？它在多线程环境中扮演什么角色？请列举其常用的阻塞方法。★★★★☆

- **是什么:** `BlockingQueue` (阻塞队列) 是一个接口, 它在标准 `Queue` 的基础上增加了**阻塞**特性。
- **目的:** 主要用于解决多线程环境下的**生产者-消费者问题**。
- **角色:**
 - 当生产者尝试向**已满**的队列中添加元素时, 它会被**阻塞**, 直到队列有空间。
 - 当消费者尝试从**已空**的队列中获取元素时, 它会被**阻塞**, 直到队列有元素。
- **常用阻塞方法:**
 - `put(e)`: 阻塞式地添加元素。
 - `take()`: 阻塞式地移除并返回元素。

2. 解释 `BlockingQueue` 是如何通过条件变量或CAS操作实现阻塞和唤醒的。★★★★☆

`BlockingQueue` 的具体实现 (如 `ArrayBlockingQueue` 和 `LinkedBlockingQueue`) 通常使用 `java.util.concurrent.locks` 包中的工具来实现阻塞和唤醒。

- `ArrayBlockingQueue`: 内部使用一个 `ReentrantLock` 和两个 `Condition` 对象 (`notEmpty` 和 `notFull`)。
 - 当队列满时, 生产者线程调用 `notFull.await()` 进入等待状态。
 - 当队列空时, 消费者线程调用 `notEmpty.await()` 进入等待状态。
 - 当消费者取走一个元素后, 会调用 `notFull.signal()` 唤醒一个生产者线程。
 - 当生产者放入一个元素后, 会调用 `notEmpty.signal()` 唤醒一个消费者线程。
- `ConcurrentLinkedQueue`: 这是一个非阻塞队列, 它使用 **CAS (Compare-And-Swap)** 原子操作来保证线程安全, 性能更高, 但它不是 `BlockingQueue`。

4. Deque 接口特性

1. 什么是 `Deque` 接口？它与 `Queue` 接口有何不同？`Deque` 如何同时实现队列和栈的功能？★★★★☆

- **是什么:** `Deque` (发音 "deck") 是 **Double Ended Queue** 的缩写, 即**双端队列**。
- **与 `Queue` 的不同:** `Queue` 只允许在队尾入队、队头出队。而 `Deque` 继承自 `Queue`, 并增加了在**队列两端**都能进行添加和删除元素的功能。
- **如何实现队列和栈:**
 - **实现队列 (FIFO):** 使用 `addLast()` (入队) 和 `removeFirst()` (出队)。
 - **实现栈 (LIFO):** 使用 `addFirst()` (入栈) 和 `removeFirst()` (出栈)。 `push()` 和 `pop()` 方法内部就是调用这两个方法。 `ArrayDeque` 是官方推荐的栈实现。

六、并发集合

1. ConcurrentHashMap 详解

1. 为什么 `HashMap` 不是线程安全的？请详细解释JDK1.7 中 `HashMap` 在并发扩容时可能导致死循环的原因。★★★★★

- **为什么不安全:** `HashMap` 的所有操作都没有同步处理，在多线程环境下并发修改会导致数据覆盖、丢失，甚至在JDK 1.7 中引发死循环。
- **JDK 1.7 死循环原因:**
 1. `HashMap` 在扩容时，需要将旧数据转移到新数组中。
 2. JDK 1.7 在转移链表时使用的是**头插法**，这会导致链表的顺序**反转**。
 3. 在多线程环境下，如果两个线程同时对同一个桶进行扩容，一个线程可能在另一个线程完成反转之前拿到链表头，导致A指向B，而B又被另一个线程反转后指向A，形成一个**循环链表**。当后续 `get` 操作遍历这个链表时，就会陷入死循环，导致CPU 100%。

2. 什么是 `ConcurrentHashMap`？请详细解释JDK1.7 和 JDK1.8 中 `ConcurrentHashMap` 的实现原理。★★★★★

- **是什么:** `ConcurrentHashMap` 是一个**线程安全的、高并发的** `Map` 实现，是 `Hashtable` 的完美替代品。
- **JDK 1.7 实现原理:**
 - **分段锁 (Segment Lock):** `ConcurrentHashMap` 内部由一个 `Segment` 数组和多个 `HashEntry` 数组组成。`Segment` 继承自 `ReentrantLock`，充当锁的角色。
 - 当需要操作数据时，不是锁住整个 `Map`，而是先定位到对应的 `Segment`，然后只**锁住这个 Segment**。
 - 这种设计大大降低了锁的粒度，允许多个线程同时访问不同 `Segment` 的数据，提高了并发度。
- **JDK 1.8 实现原理:**
 - **放弃了分段锁**，回归到类似 `HashMap` 的“数组 + 链表 / 红黑树”结构。
 - 锁的粒度进一步细化，锁住的是数组中的**每个桶的头节点**。
 - 写操作时，使用 `synchronized` 关键字锁住桶的头节点。如果头节点为空，则使用 **CAS** (Compare-And-Swap) 原子操作来尝试添加。
 - 这种设计使得锁冲突的概率更小，并发性能更高。

3. `ConcurrentHashMap` 的 `put` 方法逻辑是什么？它如何保证线程安全和高并发？★★★★☆

- **`put` 流程 (JDK 1.8):**
 1. 计算哈希值，定位到桶。
 2. 如果桶为空，则使用 **CAS** 尝试插入新节点。如果成功，操作结束。
 3. 如果 CAS 失败（说明有其他线程在此期间插入了节点），或桶不为空，则进入 `synchronized` 同步块，锁住该桶的头节点。
 4. 在同步块内，再次检查桶的状态，然后像 `HashMap` 一样，遍历链表或红黑树，进行插入或更新操作。
- **如何保证安全与高并发:**
 - **高并发:** 锁只作用于单个桶，不同桶的操作完全可以并行。

- **线程安全**: 使用 `synchronized` 保证了对同一个桶的修改是互斥的, 使用 CAS 保证了对空桶的首次插入是原子的。 `volatile` 关键字保证了节点的可见性。

4. `ConcurrentHashMap` 的 `get` 方法是否需要加锁? 为什么? ★★★★★☆

`get` 方法**不需要**加锁。

- **原因**: `ConcurrentHashMap` 内部的 `Node` 节点的 `val` 和 `next` 指针都由 `volatile` 关键字修饰。
- `volatile` 保证了当一个线程修改了节点后, 其结果能**立即对其他线程可见**。因此, 读线程总能看到最新的数据, 无需加锁。这种设计极大地提高了读取性能。

5. 为什么 `ConcurrentHashMap` 不支持 `null` 键或 `null` 值? ★★★★★☆

这是为了**避免二义性**。

- 在并发环境下, 如果 `get(key)` 返回了 `null`, 程序无法判断是“这个键不存在”, 还是“这个键存在, 但其对应的值就是 `null`”。
- 为了消除这种模糊性, `ConcurrentHashMap` 直接禁止 `null` 作为键或值。这样, 当 `get(key)` 返回 `null` 时, 含义就非常明确: 这个键在 `Map` 中不存在。

6. `ConcurrentHashMap` 是如何保证线程安全的? ★★★★★★

`ConcurrentHashMap` (JDK 1.8) 主要通过三种技术组合来保证线程安全:

1. **`volatile`**: 保证了 `Node` 数组 `table` 以及 `Node` 节点的 `val` 和 `next` 字段在多线程间的**可见性**。
2. **CAS (Compare-And-Swap)**: 用于在无锁或少锁的情况下进行原子操作, 主要用在空桶的初始化、以及扩容时设置 `fwd` 节点等场景。
3. **`synchronized`**: 用于锁定单个桶的头节点, 当发生哈希冲突时, 保证对该桶内链表或红黑树的修改是**互斥的**。

7. `ConcurrentHashMap` 在哪些地方做了并发控制? ★★★★★☆

在 JDK 1.8 中, 并发控制主要体现在以下几个关键操作上:

- **初始化数组 (`table`)**: 使用 CAS。
- **插入/更新元素 (`put`)**: 如果桶为空, 使用 CAS; 如果桶不为空, 使用 `synchronized` 锁住头节点。
- **扩容 (`resize`)**: 使用 CAS 和 `synchronized` 协同完成, 允许多个线程参与扩容, 提高效率。
- **获取大小 (`size`)**: 通过 CAS 维护一个 `baseCount` 和一个 `CounterCell` 数组来高效地进行并发计数。

8. 为什么 `ConcurrentHashMap` 在 JDK 1.8 废弃分段锁? ★★★★★☆

1. **锁粒度仍然较粗**: 即使分段, `Segment` 内部仍然是一把大锁。如果哈希分布不均, 大量操作集中在少数几个 `Segment` 上, 并发性能依然会下降。
2. **实现复杂且内存占用大**: `Segment` 继承自 `ReentrantLock`, 本身有一定开销。整个 `Segment` 数组也占用了额外的内存。
3. **`synchronized` 锁优化**: 随着 JVM 的发展, 对 `synchronized` 关键字的优化 (如锁消除、锁膨胀、自适应旋转等) 使得其性能已经不输于 `ReentrantLock`, 且使用更简单。

9. 为什么 ConcurrentHashMap 在JDK 1.8 中使用 synchronized 而不是 ReentrantLock ?

★★★★☆

1. **JVM 级别的优化**: JVM 对 `synchronized` 有深入的内置优化, 而对 `ReentrantLock` 的优化相对有限。
2. **内存占用更小**: 使用 `synchronized` 锁对象头, 不需要像 `ReentrantLock` 那样额外创建锁对象。
3. **代码更简洁**: `synchronized` 是 Java 关键字, 使用起来比 `ReentrantLock` 的 API 更简单。

2. CopyOnWriteArrayList 详解

1. 什么是 CopyOnWriteArrayList ? 它的实现原理是什么? 它适用于什么场景? 其优缺点是什么?

★★★★☆

- **是什么**: `CopyOnWriteArrayList` 是一个线程安全的 `List` 实现。
- **实现原理**: 写时复制 (Copy-On-Write)。
 - **读操作**: 直接在原数组上进行, 不加锁。
 - **写操作** (`add`, `set`, `remove`): 先加锁, 然后复制整个底层数组, 在新数组上进行修改, 最后将指向旧数组的引用原子地切换到新数组上。
- **适用场景**: 读多写少的场景。例如事件监听器列表。
- **优缺点**:
 - **优点**: 读操作性能极高, 因为完全无锁。
 - **缺点**:
 - **内存占用大**: 每次写操作都会复制整个数组。
 - **数据一致性问题**: 写操作和读操作分别在不同数组上, 读线程可能无法立即看到最新的修改, 数据存在短暂的**不一致**。

2. 如何理解 CopyOnWriteArrayList 的“写时复制”机制? ★★★★★

“写时复制”是一种**读写分离**的思想。

1. **共享读取**: 所有读线程共享访问一个不变的底层数组, 所以它们之间不需要同步。
2. **独占写入**: 当一个写线程到来时, 它会获得锁, 然后**不直接修改共享的数组**。它会创建一个该数组的**全新副本**。
3. **在新副本上修改**: 所有的修改操作都在这个私有的新副本上进行。
4. **原子替换**: 修改完成后, 写线程会原子地 (通常通过 `volatile` 和 `Unsafe`) 将 `map` 内部指向数组的引用从旧数组切换到这个新副本上。
5. 此后, 新的读线程就会访问到这个新数组了。

3. 什么是COW, 如何保证的线程安全? ★★★★★

- **COW 就是 Copy-On-Write** (写时复制) 的缩写。
- **保证线程安全**:
 1. **读写分离**: 通过复制, 保证了读操作和写操作在不同的数据结构上进行, 从根本上避免了读写冲突。
 2. **写入加锁**: 使用 `ReentrantLock` 保证了同一时刻只有一个线程能进行写操作, 避免了写写冲突。
 3. **数据不变性**: 读线程访问的数组始终是不可变的快照, 因此是绝对安全的。

3. 其他并发集合

1. 除了 ConcurrentHashMap 和 CopyOnWriteArrayList，你还知道哪些并发集合类？它们各自有什么特点和用途？★★★★☆

- CopyOnWriteArraySet: 基于 CopyOnWriteArrayList 实现的线程安全的 Set，特性类似。
- ConcurrentSkipListMap: 线程安全的、有序的 Map。底层使用跳表数据结构，支持较高的并发读写，并能提供 O(log n) 时间复杂度的操作。
- ConcurrentLinkedQueue: 线程安全的、无界的、非阻塞的队列。使用 CAS 原子操作实现，性能极高，适合高并发场景。
- 各种 BlockingQueue 实现: 如 ArrayBlockingQueue (有界、基于数组)、LinkedBlockingQueue (通常无界、基于链表)、PriorityBlockingQueue (无界、优先队列)、SynchronousQueue (不存储元素的队列)。

2. 同步容器（如Vector）的所有操作一定是线程安全的吗？★★★★☆

- 单个操作是线程安全的: Vector、Hashtable 或通过 Collections.synchronized... 包装的容器，其每个独立的方法（如 add(), get()）确实是线程安全的，因为它们都被 synchronized 修饰。
- 复合操作不是线程安全的: 如果你需要执行一系列操作，比如“检查是否包含，如果不包含则添加”，这个组合操作不是原子的。
 - 例子: `if (!vector.contains(element)) { vector.add(element); }`
 - 问题: 线程A执行完 contains() 判断为 false 后，在执行 add() 之前，线程B可能已经 add() 了同一个 element。这导致最终 vector 中出现了重复元素，违背了业务逻辑。
 - 解决方案: 对于这种复合操作，必须使用额外的、客户端侧的锁来保证其原子性，例如使用 `synchronized(vector) { ... }` 代码块。

七、集合底层数据结构与源码分析

1. 常用数据结构

1. Java集合框架中常用数据结构有哪些（如数组、链表、红黑树、哈希表）？它们各自的特点和适用场景是什么？★★★★★

数据结构	特点	优点	缺点	适用场景与代表集合
数组 (Array)	连续的内存空间，长度固定。	随机访问极快 (O(1))。	增删慢 (O(n))，因为需要移动元素；长度固定。	读多写少，需要频繁按索引访问的场景。 ArrayList。
链表 (Linked List)	非连续内存，节点通过指针相连。	增删极快 (O(1))，无需移动元素。	随机访问慢 (O(n))，需要遍历查找；内存开销更大（存储指针）。	写多读少，需要频繁插入和删除的场景。 LinkedList。
哈希表 (Hash Table)	基于哈希函数，将键映射到数组索引。	理想情况下增删查改均为 O(1)，性能极高。	哈希冲突时性能下降；通常无序。	追求极致性能，不关心顺序的键值对存储。 HashMap, HashSet。

数据结构	特点	优点	缺点	适用场景与代表集合
红黑树 (Red-Black Tree)	一种自平衡的二叉查找树。	增删查改稳定在 $O(\log n)$ ；始终保持有序。	性能不如哈希表；实现复杂。	需要一个时刻保持有序的集合。TreeMap, TreeSet。

2. HashMap 源码细节

1. 在 HashMap 中，哈希值的高位与低位异或操作 ($h \wedge (h \ggg 16)$) 的目的是什么？★★★★☆

这个操作的目的是增加哈希值的随机性，让哈希分布更均匀，从而减少哈希冲突。

- HashMap 计算桶位索引时，使用的是 $hash \& (n-1)$ ，其中 n 是数组长度。如果数组长度较小（例如16），只有哈希值的低位会参与运算，高位则完全不起作用。
- 如果两个键的 hashCode() 只有高位不同而低位相同，它们会被映射到同一个桶中，造成不必要的冲突。
- 通过将哈希码 h 右移16位 ($h \ggg 16$)，得到其高16位，再与原始哈希码 h 进行异或 ($\wedge$) 运算，就等于将高位信息也混合到了低位中。这样，即使 hashCode() 实现得不好，高位的变化也能影响到最终的索引计算，使得哈希分布更加均匀。

3. ArrayList 源码细节

1. 解释 ArrayList 中 System.arraycopy() 方法的工作原理和效率。★★★★☆

- 工作原理:** System.arraycopy() 是一个原生 (native) 方法，它不通过Java代码实现，而是直接调用底层操作系统的内存块复制功能。它能将源数组中从指定位置开始的连续一块内存，直接复制到目标数组的指定位置。
- 效率:** 因为它是原生方法，绕过了Java虚拟机的解释和编译，直接操作内存，所以其效率非常高，远快于使用Java循环逐个元素进行复制。ArrayList 的扩容、add(index, e)、remove(index) 等涉及元素移动的操作，都依赖这个高效的方法来保证性能。

4. LinkedList 源码细节

1. LinkedList 作为一个双向链表，其 Node 节点包含了哪些信息？★★★★☆

LinkedList 内部的私有静态内部类 Node<E> 包含了三个核心信息：

- E item: 当前节点存储的数据元素。
- Node<E> next: 指向下一个节点的引用。
- Node prev: 指向上一个节点的引用。

这三个字段构成了双向链表的基础，使得 LinkedList 可以在两个方向上高效遍历和操作。

5. hashCode() 与 equals()

1. Set 接口如何通过 hashCode() 和 equals() 方法保证元素的唯一性？这两个方法有什么契约？★★★★★

Set (如 HashSet) 通过两步来保证唯一性：

- 第一步：比较 hashCode()**：当添加一个元素时，首先计算其 hashCode() 值，并据此快速定位到哈希表中的一个桶。如果这个桶是空的，就直接放入。

2. **第二步：比较 equals()**：如果桶中已经有其他元素（即发生哈希冲突），则遍历桶中的所有元素，并逐个调用新元素的 equals() 方法与已有元素进行比较。如果 equals() 返回 true，说明元素已存在，添加失败；如果都返回 false，则将新元素加入。

hashCode() 与 equals() 的契约：

1. **equals 相等，hashCode 必相等**：如果两个对象通过 equals() 方法比较是相等的，那么它们的 hashCode() 方法必须返回相同的值。
2. **hashCode 相等，equals 不一定相等**：如果两个对象的 hashCode() 值相同，它们通过 equals() 比较不一定相等（这就是哈希冲突）。
3. **equals 不等，hashCode 可能相等也可能不等**。
4. **equals 自身一致性**：一个对象与自身比较 equals() 必须返回 true。
5. **hashCode 自身一致性**：在对象未被修改的前提下，多次调用 hashCode() 必须返回同一个值。

结论：hashCode() 用于快速筛选，equals() 用于最终确认。只有这两个方法都认为“相等”，Set 才会判断元素重复。

八、迭代器与并发考量

1. 迭代器类型

1. **Iterator 和 ListIterator 有什么区别？它们各自的适用范围和功能特性是什么？★★★★☆**

特性	Iterator	ListIterator
继承关系	顶层迭代器接口	继承自 Iterator
适用范围	所有 Collection (List, Set, Queue)	仅适用于 List
移动方向	只能向前 (next())	可以双向移动 (next(), previous())
索引访问	不能	可以获取当前位置的索引 (nextIndex(), previousIndex())
修改操作	只能删除 (remove())	可以增、删、改 (add(), remove(), set())

2. **Iterator 和 Enumeration 有什么区别？为什么在现代Java中更推荐使用 Iterator？**

★★★★☆

• **区别：**

1. **诞生时间**：Enumeration 是Java 1.0 的产物，Iterator 是Java 1.2 随集合框架诞生的。
2. **方法名**：Enumeration 使用 hasMoreElements() 和 nextElement()。Iterator 使用 hasNext() 和 next()。
3. **修改能力**：Enumeration 只能读，不能对集合进行任何修改。Iterator 提供了 remove() 方法。
4. **线程安全**：Enumeration 通常是与 Vector、Hashtable 等同步容器一起使用。Iterator 引入了fail-fast机制。

- **为什么推荐 `Iterator`：**

1. **功能更强：**`Iterator` 提供了 `remove()` 方法，更灵活。
2. **命名更简洁：**方法名更短，更符合现代Java的命名习惯。
3. **设计更安全：**fail-fast机制能及早暴露并发修改的错误，帮助开发者发现潜在bug。

2. 迭代器线程安全

1. 什么是快速失败 (Fail-Fast) 迭代器？它何时抛出 `ConcurrentModificationException`？其目的是什么？★★★★☆

- **是什么：**Fail-Fast 是一种迭代器机制，它在遍历集合的过程中，一旦检测到集合的结构被**从外部**（即不是通过迭代器自身的 `remove()` 或 `add()` 方法）修改了，就会**立即**抛出 `ConcurrentModificationException`。
- **何时抛出：**迭代器在创建时会记录一个 `modCount` (modification count) 的值。在每次调用 `next()` 或 `hasNext()` 时，它都会检查当前集合的 `modCount` 是否与创建时记录的值一致。如果不一致，就说明集合被外部修改了，从而抛出异常。
- **目的：**其主要目的**不是为了实现线程安全**，而是为了**及早地暴露潜在的bug**。它提醒开发者：在迭代过程中对集合进行并发修改是一种错误的行为，应该被修复。

2. 什么是安全失败 (Fail-Safe) 迭代器？它如何避免 `ConcurrentModificationException`？其优缺点是什么？★★★★☆

- **是什么：**Fail-Safe 是一种迭代器机制，它在遍历时**不会**抛出 `ConcurrentModificationException`。
- **如何避免：**它通过在**原始集合的一个副本 (snapshot)** 上进行遍历来实现。因为遍历的是副本，所以任何对原始集合的修改都不会影响到迭代过程。
- **优缺点：**
 - **优点：**不会抛出异常，程序健壮性高。
 - **缺点：**
 - **数据不一致：**迭代器看到的是集合在创建副本那一刻的状态，可能无法反映最新的修改，数据是**弱一致**的。
 - **内存开销大：**创建副本需要额外的内存和时间成本。

3. 请举例说明哪些集合是快速失败的，哪些是安全失败的。★★★★☆

- **快速失败 (Fail-Fast):**
 - `java.util` 包下的绝大多数集合，如 `ArrayList`, `LinkedList`, `HashMap`, `HashSet` 等。
- **安全失败 (Fail-Safe):**
 - `java.util.concurrent` 包下的并发集合，如 `CopyOnWriteArrayList`, `ConcurrentHashMap` 等。

4. 什么是fail-fast？什么是fail-safe？★★★★☆

- **Fail-Fast (快速失败):** 一旦检测到可能存在错误（如并发修改），就**立即、大声地失败**（抛出异常），迫使开发者正视问题。是一种“宁为玉碎，不为瓦全”的策略。
- **Fail-Safe (安全失败):** 尽可能地避免失败（不抛出异常），保证程序的继续运行，但可能以牺牲数据一致性为代价。是一种“容忍错误，继续前进”的策略。

5. 遍历的同时修改一个List有几种方式? ★★★☆☆

1. **正确方式 (推荐):** 使用迭代器自身的 `remove()` 方法。这是唯一一种在 for-each 循环结构下安全的修改方式。

```
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    if (iterator.next().equals("B")) {
        iterator.remove(); // 正确
    }
}
```

2. **正确方式 (需谨慎):** 使用传统的基于索引的 for 循环, 并在删除元素后手动调整索引。

```
for (int i = 0; i < list.size(); i++) {
    if (list.get(i).equals("B")) {
        list.remove(i);
        i--; // 关键: 索引必须回退
    }
}
```

3. **正确方式 (Java 8+):** 使用 `removeIf()` 方法, 这是最简洁、最安全的方式。

```
list.removeIf(item -> item.equals("B"));
```

4. **错误方式:** 直接在增强型 for 循环 (for-each) 中调用 `list.remove()`。这会立即导致 `ConcurrentModificationException`。

6. ConcurrentHashMap 是如何保证fail-safe的? ★★★★★☆

`ConcurrentHashMap` 的迭代器被设计为弱一致性 (weakly consistent), 而不是在严格的副本上工作。

- 当迭代器被创建时, 它反映了 `Map` 当时的某个状态。
- 在遍历过程中, 如果已有元素被修改或删除, 迭代器能够反映这些变化。
- 如果在遍历过程中有新元素被添加, 迭代器不保证能看到这些新添加的元素。
- 最重要的是, 它通过这种设计, 保证了在任何情况下都不会抛出 `ConcurrentModificationException`, 从而实现了 fail-safe 的特性。它没有采用 `CopyOnWriteArrayList` 那种重量级的完全复制策略, 而是在性能和一致性之间做了一个很好的平衡。

九、集合框架高级主题与最佳实践

1. 排序与比较

1. `Comparable` 和 `Comparator` 接口有什么区别? 如何使用它们对自定义对象进行排序? ★★★★★

这两个接口都是用来定义排序规则的, 但它们的应用场景和设计思想不同。

特性	Comparable	Comparator
定义位置	在类内部实现，作为类自身的一种属性。	在类外部定义，作为一个独立的比较器。
性质	内部比较器 或 自然排序。	外部比较器 或 自定义排序。
侵入性	高。需要修改要排序的类的源码。	低。无需修改类源码。
使用方式	一个类只能有一个自然排序规则。	可以为同一个类定义多个不同的比较器。
核心方法	<code>int compareTo(T o)</code>	<code>int compare(T o1, T o2)</code>

如何使用：

- 使用 Comparable (定义自然顺序):

让自定义类（如 Student）实现 Comparable 接口，并重写 compareTo 方法。这通常用于定义该类最主要的、默认的排序逻辑（例如按学号排序）。

```
class Student implements Comparable<Student> {
    private Integer id;
    // ... 其他属性和构造器
    @Override
    public int compareTo(Student other) {
        return this.id.compareTo(other.id); // 按id升序
    }
}
// 使用
Collections.sort(studentList);
```

- 使用 Comparator (定义临时或多种排序规则):

创建一个实现了 Comparator 接口的独立类，或者使用 Lambda 表达式。这用于实现灵活的、非默认的排序逻辑（例如按姓名、按年龄排序）。

```
// 使用独立的比较器类
class SortByAge implements Comparator<Student> {
    @Override
    public int compare(Student s1, Student s2) {
        return s1.getAge().compareTo(s2.getAge());
    }
}
// 或者使用 Lambda 表达式 (Java 8+)
studentList.sort((s1, s2) -> s1.getName().compareTo(s2.getName()));

// 使用
Collections.sort(studentList, new SortByAge());
```

2. 不可变集合

1. 如何创建不可变集合？创建不可变集合有哪些优势？ ★★★★★☆

- 如何创建:

1. `Collections.unmodifiable...()` 方法: 这是传统方式，它接收一个现有集合，并返回其只读视图。例如 `Collections.unmodifiableList(list)`。注意，这只是一个视图，如果原始集合被修改，视图也会跟着改变。
2. `List.of()`, `Set.of()`, `Map.of()` (Java 9+): 这是更现代、更推荐的方式。这些静态工厂方法直接创建一个真正意义上的不可变集合，它们在创建后无法以任何方式被修改。

- 优势:

1. 线程安全: 不可变对象本质上是线程安全的，无需任何同步措施即可在多线程间自由共享。
2. 可预测性/防御性编程: 当你把一个不可变集合传递给一个方法时，你确信它不会被意外修改，这使得程序状态更可控，减少了bug。
3. 可作为常量: 可以安全地将不可变集合定义为 `public static final` 常量。
4. 性能和内存效率: 不可变集合可以被自由共享，避免了防御性拷贝的开销。`List.of()` 等方法创建的集合通常比其可变版本有更优的内存布局。

3. 常见陷阱与优化

1. 在使用Java集合框架时，有哪些常见的陷阱和规避策略？ ★★★★★★

1. `Arrays.asList()` 的陷阱:

- 陷阱: `Arrays.asList()` 返回的不是 `java.util.ArrayList`，而是一个定长的、内部类的 `List`。对其进行 `add` 或 `remove` 操作会抛出 `UnsupportedOperationException`。
- 规避: 如果需要一个可修改的 `List`，应使用 `new ArrayList<>(Arrays.asList(array))`。

2. 在遍历时修改集合:

- 陷阱: 在 `for-each` 循环中直接调用集合的 `remove()` 方法会导致 `ConcurrentModificationException`。
- 规避: 使用迭代器自身的 `iterator.remove()` 方法，或使用 Java 8 的 `removeIf()` 方法。

3. `hashCode()` 和 `equals()` 实现不当:

- 陷阱: 将自定义对象作为 `Set` 的元素或 `Map` 的键时，如果没有正确重写 `hashCode()` 和 `equals()`，会导致元素无法按预期去重或查找。
- 规避: 始终遵守“`equals` 相等则 `hashCode` 必相等”的契约。

4. 线程安全问题:

- 陷阱: 在多线程环境中使用非线程安全的集合（如 `ArrayList`, `HashMap`）会导致数据不一致或程序崩溃。
- 规避: 使用 `java.util.concurrent` 包下的并发集合（如 `ConcurrentHashMap`），或在必要时使用同步包装器。

5. `null` 值处理:

- 陷阱: `HashMap` 允许 `null` 键值，但 `ConcurrentHashMap`, `Hashtable`, `TreeMap` 等不允许。在不同集合间传递数据时可能因 `null` 值抛出 `NullPointerException`。
- 规避: 在编码时明确 `null` 的处理策略，做好检查。

2. 如何根据实际需求选择最合适的集合类型进行性能优化? ★★★★★☆

选择集合的决策流程应遵循以下步骤:

1. 确定数据模型: 需要键值对吗?

- 是 -> `Map` 体系。
- 否 -> `Collection` 体系。

2. 确定核心需求:

- 需要保证唯一性吗? -> `Set`。
- 需要有序并允许重复吗? -> `List`。
- 需要按特定规则 (FIFO/LIFO/优先级) 处理吗? -> `Queue / Deque`。

3. 在具体体系内选择实现:

- 性能优先, 不关心顺序? -> `HashMap / HashSet` (绝大多数情况下的默认选择)。
- 需要保持插入顺序? -> `LinkedHashMap / LinkedHashSet`。
- 需要自动排序? -> `TreeMap / TreeSet`。
- 读多写少? (`List` 场景) -> `ArrayList`。
- 写多读少, 尤其是在两端操作? (`List` 场景) -> `LinkedList`。
- 需要一个栈? -> `ArrayDeque`。

4. 考虑并发:

- 需要在多线程环境中使用吗? -> 选用 `java.util.concurrent` 包下对应的并发集合。

5. 预设容量: 如果能预估元素数量, 在创建 `ArrayList` 或 `HashMap` 时指定初始容量, 可以有效避免昂贵的扩容操作。

3. 解释 `transient` 关键字在集合框架序列化中的作用。★★★★☆

`transient` 关键字用于阻止一个字段被Java的默认序列化机制序列化。

- 在 `ArrayList` 中, 其底层存储元素的数组 `elementData` 被声明为 `transient`。
- 原因: `elementData` 数组的长度通常大于实际存储的元素数量 (即 `capacity > size`), 如果直接序列化整个数组, 会浪费大量空间去存储那些未使用的 `null` 元素。
- 解决方案: `ArrayList` 重写了 `writeObject()` 和 `readObject()` 方法, 实现了自定义的序列化逻辑。在 `writeObject()` 中, 它只将 `size` 和实际存在的元素写入序列化流中, 从而达到了节省空间的目的。

4. 如何将集合变成线程安全的? ★★★★★☆

主要有两种方式:

1. 使用 `Collections` 工具类的同步包装器:

- 例如: `List<T> syncList = Collections.synchronizedList(new ArrayList<>());`
- 原理: 将非线程安全的集合包装一下, 所有方法都加上 `synchronized` 锁。
- 缺点: 性能差, 因为所有操作都竞争同一把锁, 锁的粒度太大。

2. 使用 `java.util.concurrent` 包下的并发集合 (推荐):

- 例如: `ConcurrentHashMap`, `CopyOnWriteArrayList`。

- **原理:** 采用了更先进的并发控制技术（如分段锁、CAS、写时复制），锁的粒度更细，允许多个线程在不同部分上并发操作。
- **优点:** 在保证线程安全的同时，提供了高得多的并发性能。这是现代Java并发编程的首选。

十、Stream API

1. Stream API 基础

1. Java 8中的Stream用过吗？都能干什么？ ★★★★★☆

是的，用过。Stream是Java 8引入的一个核心特性，它不是一种数据结构，而是对数据源（如集合、数组）进行计算操作的视图。

它主要能干以下几件事：

1. **函数式风格的数据处理:** 让我们能以声明式的方式（像写SQL一样）处理数据，代码更简洁、易读。
2. **链式操作:** 支持将多个操作（如 `filter`, `map`, `sorted`）链接起来，形成一个处理流水线（pipeline）。
3. **内部迭代:** Stream自己管理迭代过程，开发者只需关注要做什么（what），而不是怎么做（how）。
4. **惰性求值:** 很多Stream操作（中间操作）是惰性的，只有当终端操作被调用时，它们才会真正执行。
5. **并行处理:** 可以非常简单地通过 `.parallel()` 方法将流切换为并行流，利用多核处理器来加速计算。

例子: 从一个 `Person` 列表中，筛选出所有年龄大于18岁的，并提取他们的姓名，最后将姓名转换为大写，存入一个新的 `List`。

```
List<String> adultNames = people.stream()           // 1. 创建流
    .filter(p -> p.getAge() > 18) // 2. 筛选
    .map(Person::getName)           // 3. 映射/转换
    .map(String::toUpperCase)      // 4. 再次转换
    .collect(Collectors.toList()); // 5. 收集结果
```

2. Stream的并行流是如何实现的？ ★★★★★☆

并行流（Parallel Stream）的底层是依赖于Java 7引入的 **Fork/Join 框架**。

- **Fork (拆分):** 并行流首先会将大的数据源（通过 `Spliterator`）递归地拆分成多个更小的任务块，直到任务块足够小，可以被高效处理。
- **Join (合并):** 这些小任务块会被提交到公共的 `ForkJoinPool` 线程池中，由多个线程并发执行。每个任务执行完毕后，其结果会被逐步合并，最终形成完整的结果。

这个过程对开发者是透明的，只需调用 `parallelStream()` 或 `stream().parallel()` 即可启动。它使得利用多核CPU来提升数据处理性能变得异常简单。但是，并非所有场景都适合使用并行流，对于数据量小、或单个任务计算非常简单的场景，线程切换和管理的开销可能会超过并行带来的好处。